Python Cursus: Week 3

Leerdoelen

- ☐ CSV-bestanden inlezen en bewerken.
- ☐ Basis datacleaning: NaNs verwijderen, duplicaten verwijderen en (Boolean) filteren.
- ☐ Lineaire en kubische interpolaties toepassen op een simpele dataset.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Gebruik ChatGPT verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Bijeenkomst 3: Inlezen en verwerken van meetdata

Q1

Maak een nieuwe codecel waarin je het CSV-bestand `space_jump_synthetic.csv` inleest. Het bestand bevat een header van 4 regels en een footer van 2 regels. Lees de data in met `skiprows` en `skipfooter`, en sla de kolommen tijd, hoogte, snelheid en Mach-getal op in aparte variabelen.

Oplossing:

Cel – inlezen met `skiprows` en `skipfooter`

```
import pandas as pd

# CSV-bestand inlezen met header + footer die wegnemen worden
df = pd.read_csv(
    "space_jump_synthetic.csv",
    sep=";",
    skiprows=4,      # eerste 4 regels zijn header
    skipfooter=2,    # laatste 2 regels zijn footer
    engine="python"  # verplicht als skipfooter wordt gebruikt
)

# Kolommen opslaan in aparte variabelen
t = df["t [s]"]
h = df["h [m]"]
v = df["v [m/s]"]
mach = df["Mach [-]:"]

print(t.head())
print(h.head())
print(v.head())
print(mach.head())
```

Q2

Maak een codecel waarin je twee **subplots** maakt: boven de hoogte $h(t)$ en onder de snelheid $v(t)$, beide als functie van de tijd. Gebruik de ingelezen gegevens uit `space_jump_synthetic.csv`.

Oplossing:

Cel – subplots van hoogte en snelheid

```
import numpy as np
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

# Hoogte subplot
ax1.plot(t, h, color="blue")
ax1.set_ylabel("Hoogte h(t) [m]")
ax1.set_title("Space Jump: Hoogte en snelheid als functie van de tijd")
ax1.grid(True)

# Snelheid subplot
ax2.plot(t, v, color="red")
ax2.set_ylabel("Snelheid v(t) [m/s]")
ax2.set_xlabel("Tijd t [s]")
ax2.grid(True)

plt.tight_layout()
plt.show()
```

Q3

Maak een codecel waarin je de **potentiële energie** $E_p = mgh(t)$ en de **kinetische energie** $E_k = \frac{1}{2}mv(t)^2$ berekent (neem $m = 90\,\mathrm{kg}$, $g = 9.81\,\mathrm{m/s^2}$). Maak vervolgens één figuur waarin beide energieën als functie van de tijd worden geplot.

Oplossing:

Cel – potentiële en kinetische energie in één figuur

```
# Parameters
m = 90.0            # massa [kg]
g = 9.81            # zwaartekrachtsversnelling [m/s^2]

# Energieën
E_p = m * g * h          # potentiële energie [J]
E_k = 0.5 * m * v**2     # kinetische energie [J]

# Plot
plt.figure(figsize=(8, 5))
plt.plot(t, E_p, label="Potentiële energie $E_p$", color="blue")
plt.plot(t, E_k, label="Kinetische energie $E_k$", color="red")

plt.xlabel("Tijd t [s]")
plt.ylabel("Energie [J]")
plt.title("Potentiële en kinetische energie tijdens de sprong")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

Data filtering/cleaning

Q4

Maak een nieuwe codecel waarin je het CSV-bestand `space_jump_synthetic_nosiy.csv` inleest. Het bestand bevat een header van 4 regels en een footer van 2 regels. Lees de data in met `skiprows` en `skipfooter`, en sla de kolommen tijd, hoogte, snelheid en Mach-getal op in aparte variabelen. Het bestand bevat verder verschillende NaN's, duplicate rijen en "–1" waardes (soms wordt foute data niet als NaN genoteerd, maar als aparte waarde).

Q5

Filter de data, gebruik hiervoor de volgende code', maak vervolgens een zelfde soort grafiek van de data.

Cel – filter NaN

```
#Vervang -1 waardes door NaN waardes.
df = df.drop_duplicates()
df = df.replace(-1, np.nan)

#Filter hele dataframe op NaN waardes.
df_clean = df.dropna()

#Filter op basis van kolom.
df_clean = df.dropna(subset=["kolomnaam"])

#Check waar de NaNs zitten.
df_nan = df[df.isna().any(axis=1)]
```

Oplossing:

Cel – subplots van hoogte en snelheid

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

df = df.drop_duplicates()
df = df.replace(-1, np.nan)
df_clean = df.dropna()

t_clean = df_clean["t [s]"]
h_clean = df_clean["h [m]"]
v_clean = df_clean["v [m/s]"]
mach_clean = df_clean["Mach [-]:"]

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

# Hoogte subplot
ax1.plot(t_clean, h_clean, color="blue")
ax1.set_ylabel("Hoogte h(t) [m]")
ax1.set_title("Space Jump: Hoogte en snelheid als functie van de tijd")
ax1.grid(True)

# Snelheid subplot
ax2.plot(t_clean, v_clean, color="red")
ax2.set_ylabel("Snelheid v(t) [m/s]")
ax2.set_xlabel("Tijd t [s]")
ax2.grid(True)
```

```
plt.tight_layout()
plt.show()
```

Q6

Zoals misschien is opgevallen, staat er naast NaN-waardes en duplicaten ook *outliers* in. *Outliers* verwijderen is iets moeilijker, we moeten hiervoor namelijk zeker weten of dit niet een meting was. Vaak kan je dit door inspectie doen óf door slimmer filtering te doen. Filter de *outliers*, Zie onderstaand voorbeeld ter uitleg.

Cel – filter on values

```
#Verwijder rijen (selecteer subset) op basis van criterium dat hoogte moet zijn
dan 40 km.
df = df[df["h [m]" < 40000]
```

Oplossing:

Cel – subplots van hoogte en snelheid

```
df_clean = df_clean[(df_clean["h [m]"] < 40000) & (df_clean["h [m]"] > 1000)]
#Kan ook een nieuwe variabele df_clean2 zijn.
t_clean = df_clean["t [s]"]
h_clean = df_clean["h [m]"]
v_clean = df_clean["v [m/s]"]
mach_clean = df_clean["Mach [-]:"]

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

# Hoogte subplot
ax1.plot(t_clean, h_clean, color="blue")
ax1.set_ylabel("Hoogte h(t) [m]")
ax1.set_title("Space Jump: Hoogte en snelheid als functie van de tijd")
ax1.grid(True)

# Snelheid subplot
ax2.plot(t_clean, v_clean, color="red")
ax2.set_ylabel("Snelheid v(t) [m/s]")
ax2.set_xlabel("Tijd t [s]")
ax2.grid(True)

plt.tight_layout()
plt.show()
```

Interpolation

De hoogte van de sprong wordt slechts op discrete tijdstippen gemeten. Om de hoogte op een tussentijdse tijd t te schatten kan interpolatie worden gebruikt, waarbij wordt aangenomen dat de hoogte tussen twee opeenvolgende meetpunten continu varieert. In de eenvoudigste benadering wordt lineaire interpolatie toegepast tussen (t_1, h_1) en (t_2, h_2):

$$h(t) = h_1 + \frac{t - t_1}{t_2 - t_1}(h_2 - h_1). \tag{1}$$

Hiermee kan bijvoorbeeld de hoogte op een specifiek tijdstip tijdens zijn val worden benaderd, ook wanneer dit tijdstip niet exact gemeten is. Zie onderstaand voorbeeld:

Cel – Interpolatie voorbeeld

```
from scipy.interpolate import interp1d

v_interpolate = interp1d(t_clean, v_clean, kind="linear")

v_interpolate = f(2.5)
```

Q7

Maak een schatting van de hoogte en de snelheid op $t = 210$ seconden. Gebruik daarvoor een lineaire én een cubische interpolatie.

Oplossing:

Cel - Inschatting hoogte en snelheid

```
from scipy.interpolate import interp1d

h_interpolate_linear = interp1d(t_clean, h_clean, kind="linear")
h_interpolate_cubic = interp1d(t_clean, h_clean, kind="cubic")

print("h(210) linear: ", h_interpolate_linear(210))
print("h(210) cubic: ", h_interpolate_cubic(210))

v_interpolate_linear = interp1d(t_clean, v_clean, kind="linear")
v_interpolate_cubic = interp1d(t_clean, v_clean, kind="cubic")

print("v(210) linear: ", v_interpolate_linear(210))
print("v(210) cubic: ", v_interpolate_cubic(210))
```